

Università degli Studi di Roma Tor Vergata

Ingegneria del Software II

Software Testing e Quality Assurance su Apache Storm

Progettazione sistematica dei test,
generazione automatica e valutazione
del refactoring tramite LLM

Enrico Barbatano

Luglio 2026

Indice

1	Introduzione e selezione del SUT	2
2	Progettazione della Test Suite Manuale	2
2.1	Category Partition	2
2.2	Control-Flow Testing	6
2.3	Mutation Testing	7
3	Test Automatici	8
3.1	Randoop	8
3.2	EvoSuite	9
3.3	LLM	9
4	Valutazione del Refactoring Automatico tramite LLM	10
4.1	Obiettivo della valutazione	10
4.2	Generazione incrementale delle varianti	10
4.3	Procedura di validazione	11
4.4	Confronto dei test automatici sulle varianti	11
4.5	Risultati e interpretazione	11
5	Integrazione CI/CT e Minacce alla Validità	12
5.1	Continuous Integration e Continuous Testing	12
5.2	Minaccia alla validità: indisponibilità di EvoSuite	13
6	Conclusioni	13
A	Category Partition: RedisKeyValueStateIterator	14
B	Category Partition: StatsUtil	15
	Tabelle di progettazione dei test	16
	Materiale visivo di supporto	23
C	Prompt utilizzati per la generazione tramite LLM	25
	Tabelle del refactoring automatico	28
D	Prompt utilizzati per il refactoring tramite LLM	30

1 Introduzione e selezione del SUT

L'obiettivo del report è presentare le principali tematiche di software testing, software quality assurance e miglioramento della qualità del codice in contesti complessi, nei quali manutenibilità e testabilità rappresentano attributi critici.

La sperimentazione è stata condotta su Apache Storm, un sistema distribuito open source per l'elaborazione affidabile di flussi di dati in tempo reale. Il progetto utilizza primitive generali per il realtime processing, analogamente a quanto Hadoop offre per l'elaborazione batch.

Il codice utilizzato, le suite di test e le varianti rifattorizzate sono disponibili nel branch milestone4-refactoring del seguente repository github:

<https://github.com/enricobarbatano/storm.git>

La sperimentazione comprende la progettazione di test manuali, l'impiego di strumenti automatici e LLM, la valutazione di parametri delle classi selezionate. Il processo è supportato da una pipeline CI/CT basata su GitHub Actions e Maven.

Selezione del SUT

Applicando i criteri di selezione forniti ed escludendo le classi banali, sono stati individuati due SUT con caratteristiche opposte:

- **RedisKeyValueStateIterator**: classe compatta di 69 LOC, dedicata all'iterazione su coppie chiave-valore memorizzate in Redis;
- **StatsUtil**: classe utility di 1.709 LOC, caratterizzata da elevata complessità e da 265 code smell iniziali.

Il ruolo, le responsabilità e le funzionalità delle due classi sono approfonditi nelle sezioni successive, in relazione alla progettazione delle rispettive suite di test.

2 Progettazione della Test Suite Manuale

2.1 Category Partition

L'obiettivo di questa fase è progettare una suite di test a partire dalle specifiche funzionali dei SUT, considerando principalmente il comportamento osservabile delle classi. La tecnica Category Partition è stata utilizzata per suddividere i domini di input e gli stati rilevanti in classi di equivalenza, evitando di provare tutte le combinazioni possibili.

In assenza di una documentazione funzionale esterna completa, la ricostruzione delle specifiche è stata condotta mediante una procedura articolata in quattro passaggi:

1. descrizione del ruolo della classe nel sistema;
2. analisi semantica dei Javadoc e delle firme dei metodi;
3. selezione delle funzionalità rappresentative delle principali responsabilità della classe;
4. partizionamento dei relativi domini di input e degli stati osservabili.

La descrizione preliminare del ruolo delle classi è necessaria per motivare la selezione delle funzionalità. Nel caso di una classe stateful, la progettazione deve considerare il ciclo di vita dell'oggetto e l'interazione con le

risorse esterne. Nel caso di una classe utility, invece, è necessario individuare un campione rappresentativo dei domini di trasformazione implementati dai numerosi metodi disponibili.

Analisi e progettazione di `RedisKeyValueStateIterator`

Ruolo e responsabilità della classe `RedisKeyValueStateIterator` è un iteratore specializzato per il recupero di coppie chiave-valore memorizzate in uno stato persistente Redis. A differenza di un iteratore che opera interamente in memoria, la classe è stateful e non dispone immediatamente dell'intero insieme di elementi.

I dati vengono recuperati progressivamente mediante il comando `hscan`, che restituisce un insieme limitato di coppie e un cursore utilizzato per continuare la scansione. La classe deve quindi coordinare tre tipi di stato:

- lo stato remoto della scansione Redis, rappresentato dal cursore;
- lo stato locale del chunk corrente, che può essere vuoto, parzialmente consumato oppure completamente consumato;
- lo stato della risorsa Redis, che deve essere acquisita e restituita correttamente anche in presenza di errori.

La correttezza della classe non dipende soltanto dagli elementi restituiti, ma anche dalla capacità di stabilire quando la scansione sia realmente conclusa e di non lasciare risorse Redis acquisite.

Motivazione della selezione delle funzionalità Dall'analisi dei Javadoc di `loadChunkFromStateStorage` e `isEndOfDataFromStorage`, riportati rispettivamente nelle Figure 1 e 2, sono state selezionate tre funzionalità principali:

- **F1 – Caricamento incrementale dei dati:** verifica del recupero di un chunk tramite `hscan` e della restituzione degli elementi ricevuti;
- **F2 – Riconoscimento della fine dei dati:** verifica della relazione tra cursore Redis, contenuto del chunk e stato di consumo dell'iteratore;
- **F3 – Gestione della risorsa Redis:** verifica dell'acquisizione e della restituzione dell'istanza Redis sia nel flusso normale sia nel flusso eccezionale.

Queste funzionalità sono state scelte perché rappresentano il ciclo di vita principale dell'iteratore. F1 riguarda l'acquisizione dei dati, F2 controlla la continuazione o la conclusione dell'iterazione, mentre F3 garantisce la sicurezza dell'interazione con la dipendenza esterna.

Partizionamento del dominio Il dominio è stato partizionato considerando sia gli input ricevuti da Redis sia lo stato osservabile dell'iteratore. La definizione completa delle categorie A–F è riportata nell'Appendice A.

- **Categoria A – Dati restituiti da Redis:** distingue una lista vuota, un singolo elemento, più elementi, un risultato nullo e un chunk contenente esattamente `chunkSize` elementi. Questa categoria verifica F1;
- **Categoria B – Stato della scansione:** distingue il cursore finale, il cursore non finale, la scansione composta da più chunk e il caso di un chunk vuoto con cursore non finale. Questa categoria verifica F2;
- **Categoria C – Stato osservabile dell'iteratore:** distingue l'assenza di elementi, la presenza di uno o più elementi e le condizioni precedenti e successive al loro consumo. Questa categoria verifica la relazione tra il chunk locale e F2;
- **Categoria D – Dati pendenti locali:** considera il caso in cui gli iteratori `pendingPrepare` e `pendingCommit` siano vuoti, così da isolare il comportamento dello storage Redis;

- **Categoria E – Esito dell'accesso a Redis:** distingue la conclusione corretta di `hscan` dal sollevamento di una `RuntimeException`;
- **Categoria F – Gestione della risorsa:** verifica l'acquisizione dell'istanza Redis e la chiamata a `returnInstance` sia dopo una lettura corretta sia in presenza di un'eccezione.

Progettazione dei casi di test Le partizioni astratte sono state tradotte in dieci casi di test. Le dipendenze `RedisCommands`, `RedisCommandsInstanceContainer` e `ScanResult` sono state simulate mediante Mockito, evitando la necessità di utilizzare un'istanza Redis reale e rendendo riproducibili risposte vuote, risultati nulli, cursori intermedi ed eccezioni.

Alla quantità di elementi restituiti è stata applicata la Boundary Value Analysis. Impostando: `chunkSize=2` sono stati verificati i seguenti valori:

- 0 elementi, cioè storage o chunk vuoto;
- 1 elemento, cioè caso minimo non vuoto;
- 2 elementi, cioè chunk pieno.

È stata adottata una selezione unidimensionale: ciascuna partizione significativa viene esercitata almeno una volta, senza generare il prodotto cartesiano di tutte le combinazioni tra quantità di dati, cursore, stato dell'iteratore ed esito dell'accesso a Redis.

La matrice finale della suite Category Partition è riportata nella Tabella 1.

Analisi e progettazione di StatsUtil

Ruolo e responsabilità della classe `StatsUtil` è una God Class utility composta da metodi statici e caratterizzata da oltre 1.700 LOC. La classe svolge il ruolo di motore di trasformazione e aggregazione per il sistema di monitoraggio di Apache Storm.

I metodi ricevono dati grezzi prodotti dai componenti della topologia, in particolare Bolt e Spout, e li trasformano in statistiche aggregate utilizzate dalla UI e dalle API di monitoraggio. Le elaborazioni considerate comprendono l'aggregazione di contatori organizzati per finestra temporale e stream, il calcolo di medie pesate e la combinazione di conteggi e latenze specifiche dei componenti Bolt e Spout.

A differenza di `RedisKeyValueStateIterator`, `StatsUtil` non possiede un unico ciclo di vita osservabile. La classe raccoglie numerose funzioni statiche che operano su mappe annidate, liste, contatori, latenze e oggetti Thrift. Data l'ampiezza del SUT, una copertura black-box completa di tutti i metodi non sarebbe risultata compatibile con una suite Category Partition contenuta. È stato quindi selezionato un campione rappresentativo delle principali trasformazioni implementate dalla classe.

Motivazione della selezione delle funzionalità L'analisi dei contratti Javadoc ha portato all'individuazione di quattro funzionalità principali:

- **F1 – Aggregazione di contatori:** somma di valori organizzati in strutture del tipo `window -> stream -> value`, sia all'interno di una singola mappa sia tra più mappe;
- **F2 – Calcolo di medie pesate:** combinazione di valori medi e conteggi, includendo il caso limite di un conteggio pari a zero;
- **F3 – Aggregazione delle statistiche Bolt:** combinazione del conteggio `executed` con le latenze di esecuzione e di processo, sia a livello complessivo sia per singola stream;
- **F4 – Aggregazione delle statistiche Spout:** combinazione del conteggio `acked` con la `complete latency`, sia a livello complessivo sia per singola stream.

Queste funzionalità sono state scelte perché rappresentano livelli crescenti di complessità presenti nella classe. F1 verifica le operazioni di base sulle mappe annidate; F2 verifica la correttezza e la stabilità matematica delle medie pesate; F3 e F4 verificano aggregazioni più articolate e specifiche dei principali componenti della topologia Storm.

Partizionamento del dominio Le partizioni sono state definite considerando la forma delle strutture dati in ingresso, i valori numerici contenuti nelle mappe, le condizioni necessarie al calcolo delle medie pesate e le statistiche specifiche di Bolt e Spout. La definizione completa è riportata nell'Appendice B.

- **Categoria A – Forma dell'input:** distingue input nullo, input vuoto, una singola finestra, una singola stream, più stream e chiavi presenti in una o più mappe;
- **Categoria B – Valori numerici:** distingue valori positivi, valore zero, valori mancanti e valori rappresentati mediante Long o Double;
- **Categoria C – Medie pesate:** distingue conteggio positivo, conteggio pari a zero e presenza di più stream nella stessa finestra temporale;
- **Categoria D – Statistiche Bolt:** considera il conteggio `executed`, la `execute latency`, la `process latency` e le statistiche associate a una stream Bolt;
- **Categoria E – Statistiche Spout:** considera il conteggio `acked`, la `complete latency` e le statistiche associate a una stream Spout.

Progettazione dei casi di test La suite `Category Partition` di `StatsUtil` è composta da quattordici casi di test. I casi TC01–TC05 verificano l'aggregazione dei contatori, TC06–TC08 il calcolo delle medie pesate, TC09–TC11 le statistiche Bolt e TC12–TC14 le statistiche Spout.

La Boundary Value Analysis è stata applicata ai confini relativi alla cardinalità delle strutture dati e ai valori numerici. In particolare:

- TC01 e TC06 verificano la cardinalità minima non vuota, utilizzando una sola finestra temporale e una sola stream;
- TC02 e TC07 verificano il valore immediatamente successivo alla cardinalità minima, utilizzando due stream nella stessa finestra;
- TC03 verifica il limite numerico costituito da un contatore pari a zero;
- TC08 verifica il caso limite di un conteggio pari a zero nel calcolo della media pesata, controllando che non si producano divisioni per zero o risultati numerici non validi;
- TC14 verifica il limite inferiore relativo alla cardinalità delle strutture, utilizzando due mappe di input vuote.

Il caso TC10, basato su input `null`, rappresenta invece una partizione di robustezza relativa all'assenza delle strutture di input e non un valore limite numerico. Gli altri test coprono classi di equivalenza relative alla presenza delle metriche, alla distribuzione delle chiavi tra più mappe e al tipo di statistica aggregata.

Poiché le mappe annidate generano un numero elevato di combinazioni, è stata adottata una selezione unidimensionale. Ogni partizione significativa viene coperta almeno una volta, senza esplorare il prodotto cartesiano di tutte le combinazioni tra forma dell'input, valori numerici, finestre, stream e tipologia del componente.

La matrice di progettazione è riportata nella Tabella 2.

Risultati

La prima valutazione JaCoCo ha evidenziato una differenza rilevante tra le due suite. Per `RedisKeyValueStateIterator` sono stati ottenuti l'83% di Instruction Coverage e l'87% di Branch Coverage. Per `StatsUtil`, invece, sono stati raggiunti il 9% di Instruction Coverage e l'11% di Branch Coverage.

La differenza è coerente con il ruolo delle classi. La suite Redis copre il ciclo di vita principale di una classe compatta e focalizzata. La suite di `StatsUtil`, invece, copre soltanto un campione delle responsabilità presenti in una God Class di oltre 1.700 LOC.

Anche PIT ha confermato questo comportamento. La suite Category Partition di `RedisKeyValueStateIterator` ha ottenuto una Mutation Coverage del 67%, uccidendo 8 mutanti su 12. Per `StatsUtil`, la Mutation Coverage è stata pari al 10%, con 42 mutanti uccisi su 413.

La Coverage più contenuta, soprattutto per `StatsUtil`, evidenzia invece la presenza di metodi e rami non ancora esercitati.

Per questo motivo è stata introdotta una successiva fase di Adequacy Growth. I report JaCoCo sono stati utilizzati per individuare metodi e rami scoperti da raggiungere tramite Control-Flow Testing; i report PIT sono stati invece utilizzati per rafforzare gli oracoli rispetto ai mutanti sopravvissuti.

2.2 Control-Flow Testing

La fase di Control-Flow Testing è stata introdotta come attività di *Adequacy Growth*, con l'obiettivo di estendere la suite iniziale basata su Category Partition. La suite black-box aveva già validato le principali funzionalità osservabili dei SUT, ma non garantiva una buona copertura in termini di instruction e branch coverage.

Per individuare i nuovi casi di test è stato analizzato il report JaCoCo prodotto dopo l'esecuzione delle suite Category Partition. Il report fornisce informazioni strutturali utili, tra cui metodi non coperti, istruzioni non eseguite, branch mancanti e percentuali di Instruction e Branch Coverage. Tali informazioni sono state utilizzate come guida per decidere quali parti del codice richiedessero ulteriori test. Le evidenze di partenza sono riportate nelle Figure 6 e 7.

A differenza della fase precedente, questa fase adotta quindi un approccio prevalentemente *white-box*. La progettazione dei test è infatti guidata dalla struttura interna del codice e dai gap di copertura evidenziati da JaCoCo. Questa scelta è motivata dal diverso obiettivo della fase: non definire nuove partizioni funzionali, ma raggiungere in modo mirato metodi e rami non esercitati dalla suite black-box iniziale.

Control-Flow Testing: `RedisKeyValueStateIterator`

Nel caso di `RedisKeyValueStateIterator`, il report JaCoCo ha mostrato che i metodi legati al caricamento dei chunk e alla gestione del cursore erano già esercitati dalla suite Category Partition, mentre rimanevano scoperti alcuni metodi protetti di supporto. In particolare, sono stati selezionati `decodeKey`, `decodeValue` e `isTombstoneValue`, perché rappresentano percorsi logici non attraversati dal normale flusso di caricamento da Redis.

I test sono stati quindi progettati per forzare direttamente l'esecuzione di questi metodi, mantenendo comunque oracoli semplici e deterministici: La progettazione dei casi di test è riassunta nella Tabella 3.

Control-Flow Testing: `StatsUtil`

Nel caso di `StatsUtil`, il report JaCoCo ha evidenziato un numero elevato di metodi statici non ancora esercitati. Questo risultato è coerente con la natura della classe, che contiene numerose utility e funzioni di trasformazione dati. La selezione dei test Control-Flow è stata quindi orientata verso metodi pubblici rimasti scoperti o poco coperti, privilegiando quelli rappresentativi di funzionalità diverse rispetto alla suite iniziale.

Sono stati inclusi test per utility semplici, classificazione dei componenti, conversioni di heartbeat e statistiche, aggregazioni complete di stream, calcolo di medie pesate su più executor, filtraggio degli stream di sistema ed estrazione di informazioni host/port. Ogni test è stato progettato per raggiungere uno specifico metodo o ramo individuato tramite JaCoCo, evitando combinazioni inutilmente complesse. La progettazione dei casi di test è riportata nella Tabella 4.

Risultati della fase Control-Flow

L'esecuzione congiunta delle suite Category Partition e Control-Flow ha prodotto un incremento della copertura strutturale. Per `RedisKeyValueStateIterator`, JaCoCo ha riportato il **100% di Instruction Coverage**, mentre la **Branch Coverage è rimasta pari all'87%**. Anche PIT mostra un miglioramento: la **Mutation Coverage passa dal 67% al 92%**, con **11 mutanti uccisi su 12**.

Per `StatsUtil`, la copertura JaCoCo è aumentata dal **9% al 21% di Instruction Coverage** e dall'**11% al 26% di Branch Coverage**. Anche PIT evidenzia un miglioramento: la **Mutation Coverage passa dal 10% al 23%**, con **94 mutanti uccisi su 413**. I report PIT dopo l'integrazione dei test Control-Flow sono riportati nelle Figure 9 e 11.

Questi risultati confermano che i test Control-Flow hanno aumentato l'adeguatezza della suite, raggiungendo metodi e rami non coperti dalla sola Category Partition. Nel caso di `StatsUtil`, tuttavia, la copertura resta limitata dalla grande dimensione della classe e dall'elevato numero di metodi statici ancora non esercitati.

2.3 Mutation Testing

La fase di Mutation Testing è stata introdotta dopo Category Partition e Control-Flow Testing per valutare non solo quante istruzioni vengono eseguite, ma quanto gli assert siano realmente capaci di rilevare comportamenti errati. A questo scopo è stato utilizzato PIT, che genera versioni artificialmente modificate del codice, dette *mutanti*. Un mutante rappresenta una piccola alterazione del programma originale, ad esempio la sostituzione di un valore di ritorno booleano, la rimozione di una chiamata a metodo o la modifica di un'operazione aritmetica. Se almeno un test fallisce, il mutante viene considerato *ucciso*; se invece tutti i test continuano a passare, il mutante è considerato *sopravvissuto*.

I risultati ottenuti dopo Category Partition e Control-Flow Testing hanno mostrato un incremento dell'adeguatezza, ma anche la presenza di mutanti ancora vivi. È stata quindi progettata una nuova suite mirata, basata sull'analisi dei report PIT. I mutanti sopravvissuti sono stati utilizzati per individuare comportamenti raggiunti ma non verificati da oracoli sufficientemente forti, oppure condizioni non ancora esercitate dalle suite precedenti.

Mutation Testing: `RedisKeyValueStateIterator`

Nel caso di `RedisKeyValueStateIterator`, il report PIT ha evidenziato un mutante sopravvissuto nel metodo `isTombstoneValue`. Come mostrato in Figura 12, la mutazione sostituiva il valore di ritorno booleano con `false`. Questo indicava che le suite precedenti non verificavano esplicitamente il caso positivo del tombstone: il metodo veniva esercitato, ma non esisteva ancora un assert in grado di fallire se un vero tombstone non fosse stato riconosciuto.

Per uccidere questo mutante è stato quindi aggiunto un test mirato che costruisce il valore tombstone tramite `DefaultStateEncoder` e verifica che `isTombstoneValue` restituisca `true`. Questa scelta rafforza l'oracolo sul comportamento semantico del metodo, distinguendo esplicitamente il valore ordinario dal marcatore di cancellazione. Il test progettato è riassunto nella Tabella 5.

Mutation Testing: StatsUtil

Per `StatsUtil`, l'analisi dei report PIT ha mostrato un numero maggiore di mutanti sopravvissuti, coerentemente con la dimensione della classe e con la presenza di molti metodi statici. I mutanti vivi riguardavano principalmente valori di ritorno sostituiti, mappe o liste vuote, calcoli aritmetici, medie pesate, aggregazioni Bolt/Spout, merge di statistiche e gestione di casi limite.

La suite `StatsUtilMutationTest` è stata quindi progettata per colpire questi punti specifici. I test sono stati aggiunti a partire dai comportamenti alterati rilevati da PIT: modificabilità delle strutture restituite, correttezza delle somme, calcolo dei totali pesati, gestione di valori mancanti, filtraggio degli stream di sistema e aggregazioni complesse. La mappatura tra test e mutazioni target è riportata nella Tabella 6.

Risultati

Dopo l'introduzione del test mirato, PIT ha riportato per `RedisKeyValueStateIterator` una **Mutation Coverage del 100%**, con **12 mutanti uccisi su 12**, come mostrato in Figura 13. Questo indica che tutti i mutanti generati sulla classe sono stati raggiunti e rilevati dalla suite completa.

Per `StatsUtil`, dopo l'aggiunta della suite di Mutation Testing, PIT ha riportato una **Mutation Coverage del 38%**, con **156 mutanti uccisi su 413**, come mostrato in Figura 14. Il risultato mostra un miglioramento rispetto alla fase Control-Flow, pur confermando che la classe rimane difficile da coprire completamente a causa della sua ampiezza e della quantità di logica non ancora esercitata.

3 Test Automatici

Completata la progettazione delle suite manuali, cioè Category Partition, Control-Flow Testing e Mutation Testing, sono stati introdotti strumenti di generazione automatica con l'obiettivo di complementare la validazione manuale. Tali strumenti non sostituiscono la progettazione sistematica dei test, ma forniscono una verifica aggiuntiva della robustezza del codice e della neutralità semantica del refactoring.

In particolare, i test automatici sono stati utilizzati come suite di regressione, come supporto all'aumento della copertura e come base per un confronto sperimentale tra approcci diversi di generazione dei test.

3.1 Randoop

Dopo la progettazione delle suite manuali, Randoop è stato introdotto come strumento automatico di supporto alla validazione del refactoring. Il suo obiettivo principale non è sostituire i test progettati tramite Category Partition, Control-Flow e Mutation Testing, ma fornire una suite di regressione generata automaticamente sulla classe originale.

Per entrambe le classi, Randoop è stato configurato per generare test in stile JUnit 4, con un limite massimo di 200 test. I test generati sono stati poi filtrati manualmente, eliminando quelli instabili, ridondanti o non significativi.

RedisKeyValueStateIterator

Per `RedisKeyValueStateIterator`, la generazione diretta ha inizialmente prodotto una copertura pari a 0%. Il risultato dipende da due limiti strutturali: i metodi principali sono `protected` e la classe richiede dipendenze `Redis/Jedis` non istanziabili automaticamente in modo utile da Randoop.

Per rendere il SUT esplorabile è stata quindi introdotta una *harness*, cioè una classe di supporto che espone i metodi protetti e fornisce dipendenze controllate.

StatsUtil

Per `StatsUtil` non è stata necessaria una *harness*, poiché la classe è composta da metodi statici direttamente invocabili. Tuttavia, la complessità degli input richiesti, basati su mappe annidate e oggetti Thrift, ha limitato l'efficacia della generazione casuale.

Molti test prodotti da Randoop esercitavano casi accessori o poco rappresentativi e sono stati quindi scartati. Questo evidenzia che l'assenza di problemi di visibilità non garantisce una buona esplorazione automatica quando la classe richiede input semanticamente complessi.

Risultati Dopo l'introduzione della *harness* e il filtraggio dei test non significativi, la suite Randoop selezionata per `RedisKeyValueStateIterator` ha raggiunto il **57% di Instruction Coverage** e il **25% di Branch Coverage**.

Per `StatsUtil`, invece, la suite finale ha raggiunto il **7% di Instruction Coverage** e il **4% di Branch Coverage**.

Nel complesso, Randoop si è dimostrato utile come baseline automatica e come supporto al regression testing. I risultati confermano tuttavia che la generazione casuale deve essere affiancata da test manuali progettati sistematicamente.

3.2 EvoSuite

L'adozione di EvoSuite aveva come scopo primario la massimizzazione automatica dell'adeguatezza strutturale, cioè Instruction e Branch Coverage, attraverso una tecnica di generazione basata su algoritmi evolutivi che operano direttamente a livello di bytecode Java.

L'obiettivo era ottenere una suite complementare a quella manuale, capace di esplorare sistematicamente i rami logici delle classi e di fornire un ulteriore termine di confronto rispetto a Randoop e ai test generati tramite LLM.

Tuttavia, l'attività ha riscontrato limitazioni tecniche significative. EvoSuite è riuscito a generare file di test, come mostrato in Figura 15, ma tali test non sono risultati compilabili all'interno dell'ambiente Apache Storm 2.7.1. Il problema non riguarda quindi l'assenza totale di generazione, ma l'impossibilità di integrare correttamente la suite prodotta nel progetto reale.

Di conseguenza, EvoSuite è stato mantenuto nell'analisi come evidenza sperimentale dei limiti degli strumenti automatici su progetti legacy complessi. Per una descrizione più dettagliata delle criticità tecniche, dei problemi di classpath e delle barriere di compilazione che hanno impedito il pieno svolgimento di questa fase, si rimanda al Paragrafo 5.

3.3 LLM

L'uso di Microsoft Copilot è stato finalizzato a valutare l'efficacia degli LLM nella generazione automatica di test complessi tramite diverse strategie di prompt engineering. Sono stati confrontati tre approcci a complessità crescente: **zero-shot**, **few-shot** e **guided Tree-of-Thought**. I prompt utilizzati per le tre strategie sono riportati nell'Appendice C.

Nel caso dello **zero-shot prompting**, al modello sono stati forniti solo il codice sorgente della classe e un'istruzione generale per generare una suite JUnit. Questo approccio permette di valutare la capacità del modello di derivare automaticamente i casi di test a partire dal solo sorgente.

Nel caso del **few-shot prompting**, il prompt è stato arricchito con esempi di test manuali. L'obiettivo era guidare il modello verso uno stile più coerente con le suite già progettate, sia nella struttura degli assert sia nella scelta dei casi limite.

Infine, il **guided Tree-of-Thought** ha guidato il modello attraverso una sequenza più strutturata: identificazione dei metodi, generazione di casi base, individuazione di edge case ed eccezioni, e costruzione finale di una suite completa. Questo approccio è stato usato per verificare se una maggiore guida nel ragionamento producesse test più efficaci.

RedisKeyValueStateIterator

Per la classe `RedisKeyValueStateIterator`, l'approccio basato su LLM si è dimostrato particolarmente efficace. Tutte e tre le strategie di prompting hanno raggiunto il **100% di Instruction Coverage** e il **100% di Branch Coverage**. Anche i risultati PIT sono rimasti stabili tra i prompt, con una **Mutation Coverage del 92%**, pari a **11 mutanti uccisi su 12**. I risultati completi sono riportati nella Tabella 7.

Questo risultato indica che, su una classe piccola e semanticamente circoscritta, l'LLM è stato in grado di generare test molto efficaci indipendentemente dalla strategia di prompting. In particolare, il modello ha coperto anche aspetti specifici del dominio, come la gestione dei valori tombstone e il comportamento dell'iteratore rispetto alla fine dei dati.

StatsUtil

Nel caso di `StatsUtil`, i risultati sono stati più contenuti, ma comunque significativi. Tutte le strategie hanno raggiunto una **Instruction Coverage del 23%**. La **Branch Coverage** si è attestata al **30%** con zero-shot e al **31%** con few-shot e guided Tree-of-Thought. Anche il Mutation Testing conferma questa tendenza: zero-shot ha ottenuto una **Mutation Coverage del 27%**, con **113 mutanti uccisi su 413**, mentre few-shot e guided Tree-of-Thought hanno raggiunto una **Mutation Coverage del 28%**, con **116 mutanti uccisi su 413**. Il confronto completo è riportato nella Tabella 8.

Questi risultati mostrano che, su una classe molto ampia e complessa, l'aggiunta di esempi o di una guida strutturata migliora leggermente la qualità degli oracoli e la capacità di uccidere mutanti, ma non determina un salto significativo nella copertura complessiva. Il limite principale resta la complessità degli input richiesti da `StatsUtil`, basati su strutture dati annidate, oggetti Thrift e vincoli semantici difficili da ricostruire automaticamente.

4 Valutazione del Refactoring Automatico tramite LLM

4.1 Obiettivo della valutazione

L'obiettivo di questa fase è valutare la capacità di un Large Language Model, nello specifico Microsoft Copilot, di eseguire automaticamente il refactoring di classi Java reali senza alterarne la semantica. Il refactoring non è stato quindi valutato soltanto in base alla riduzione dei code smell, ma anche rispetto alla capacità delle varianti prodotte di compilare, sostituire la classe originale e continuare a soddisfare gli stessi oracoli di test.

Per ciascuna classe è stata considerata una versione originale, indicata con C_0 , e sono state generate quattro varianti rifattorizzate, C_1-C_4 . Le varianti sono state ottenute fornendo all'LLM un contesto progressivamente più ampio e approfondito. Lo scopo di questa strategia era verificare se l'aggiunta di informazioni statiche e dinamiche permettesse al modello di produrre refactoring più affidabili e coerenti rispetto ai comportamenti verificati dalle suite di test. I prompt completi sono riportati nell'Appendice D.

4.2 Generazione incrementale delle varianti

La generazione ha seguito una strategia incrementale nella quale ogni variante aggiunge nuovi vincoli rispetto alla precedente.

La variante C_1 è stata generata utilizzando il sorgente originale e l'elenco dei code smell rilevati da SonarCloud. Il Prompt D 1 richiede interventi strettamente limitati agli smell indicati e impone di conservare package, API pubbliche e firme dei metodi.

La variante C_2 aggiunge al contesto la suite Black-Box progettata tramite Category Partition. Il Prompt D 2 usa tali test come specifica eseguibile del comportamento osservabile, con l'obiettivo di impedire che la rimozione degli smell modifichi le funzionalità verificate nelle diverse partizioni di input.

La variante C_3 aggiunge i test Control-Flow e le informazioni di coverage prodotte da JaCoCo. Il Prompt D 3 orienta il modello verso i rami e i casi limite individuati durante l'Adequacy Growth, chiedendo di preservare la logica dei percorsi decisionali complessi.

La variante C_4 utilizza infine il contesto più completo, includendo i risultati PIT e i test progettati per uccidere i mutanti sopravvissuti. Il Prompt D richiede che la variante continui a soddisfare gli oracoli rafforzati durante il Mutation Testing e rimanga un *drop-in replacement* di C_0 .

In questo modo, le quattro versioni non rappresentano quattro refactoring indipendenti, ma quattro livelli di contesto fornito all'LLM. Il confronto permette di valutare se gli oracoli progressivamente aggiunti influenzino sia la qualità statica del codice sia il comportamento delle suite automatiche generate sulle varianti.

4.3 Procedura di validazione

Ogni variante è stata integrata nel medesimo package della classe originale e sottoposta alla stessa procedura di verifica. In primo luogo è stata eseguita la compilazione Maven, comprendente i controlli Checkstyle. Successivamente è stata eseguita la suite standard del progetto, utilizzata come oracolo di regressione e come verifica della sostituibilità della variante. Si è mostrato che le varianti di entrambe le classi compilano e superano la suite standard, come riportato nella Tabella 9.

4.4 Confronto dei test automatici sulle varianti

La valutazione non si è limitata al codice rifattorizzato. Per ogni variante sono state considerate anche nuove suite automatiche LLM CoT e Randoop, al fine di verificare se il cambiamento strutturale della classe modificasse l'adequatezza dei test generati.

La suite LLM è stata prodotta con la strategia CoT già descritta nell'Appendice C, fornendo al modello il sorgente della variante corrente. Randoop è stato invece eseguito con la stessa configurazione utilizzata per la baseline e con un limite massimo di 200 test. I test duplicati, instabili o limitati a eccezioni banali su input `null` sono stati esclusi dalla suite finale. EvoSuite non è risultato utilizzabile nell'ambiente Apache Storm 2.7.1 e viene quindi indicato come N/A.

Per rendere il confronto omogeneo, i risultati delle varianti sono espressi come delta rispetto ai valori ottenuti dalla stessa suite sulla classe originale C_0 . Per ogni metrica M , il delta è calcolato come:

$$\Delta M(C_i, S) = M(C_i, S) - M(C_0, S)$$

dove C_i è la variante considerata e S identifica l'approccio di generazione della suite, cioè LLM CoT oppure Randoop. Un valore positivo indica un miglioramento rispetto alla baseline della stessa suite; un valore negativo indica una diminuzione.

4.5 Risultati e interpretazione

La riduzione dei code smell è sintetizzata nella Tabella 10. `RedisKeyValueStateIterator` passa da uno smell in C_0 a zero smell in tutte le varianti. `StatsUtil` passa da 265 smell in C_0 a 19 in C_1 , 45 in C_2 , 19 in C_3

e 19 in C_4 . La riduzione è quindi rilevante in tutte le versioni, sebbene C_2 conservi un numero maggiore di rilievi statici.

I delta di `RedisKeyValueStateIterator` sono riportati nella Tabella 11. I risultati sono sostanzialmente stabili tra le quattro varianti. La suite LLM conserva la piena Instruction Coverage, perde 12,50 punti percentuali di Branch Coverage e migliora il Mutation Score di 8,33 punti. Randoop non mostra variazioni rispetto alla propria baseline. L'aggiunta progressiva di contesto non determina quindi differenze misurabili tra C_1 e C_4 .

I delta di `StatsUtil` sono riportati nella Tabella 12. Rispetto alla baseline LLM CoT, C_1 migliora di 32 punti nelle istruzioni, 23 nei branch e 22 nel Mutation Score; C_2 ottiene l'incremento maggiore, pari a 55 punti nelle istruzioni, 33 nei branch e 23 nel Mutation Score. C_3 mantiene miglioramenti più contenuti, mentre C_4 rimane vicina alla baseline, con incrementi di 2 punti nelle istruzioni, 1 nei branch e nessuna variazione del Mutation Score.

Anche rispetto alla baseline Randoop i delta sono non negativi, ma di ampiezza molto inferiore. C_1 rappresenta il miglior caso, con un incremento di 10 punti nelle istruzioni, 7 nei branch e 6 nel Mutation Score. Per C_2 , C_3 e C_4 , le variazioni sono limitate a pochi punti percentuali.

Questi risultati indicano che il refactoring può influenzare l'adeguatezza dei test automatici anche quando la semantica osservata dalla suite standard rimane invariata. Una possibile spiegazione è che modifiche strutturali, pur semanticamente neutrali, rendano alcuni percorsi più o meno accessibili ai generatori. Il fenomeno è particolarmente evidente per i test LLM su `StatsUtil`, mentre Randoop rimane generalmente poco sensibile alle differenze tra le varianti.

C_2 mostra inoltre che la riduzione degli smell e l'adeguatezza dinamica rappresentano dimensioni differenti. Pur conservando 45 smell, C_2 ottiene i delta migliori con la suite LLM. Non è quindi possibile concludere che la variante con meno smell sia automaticamente quella più facile da testare o quella con il Mutation Score maggiore.

5 Integrazione CI/CT e Minacce alla Validità

5.1 Continuous Integration e Continuous Testing

Il progetto è stato integrato con un meccanismo di **Continuous Integration e Continuous Testing (CI/CT)** basato su **GitHub Actions**. A ogni push sul branch `milestone4-refactoring`, il workflow esegue automaticamente la compilazione e le suite di test dei moduli interessati mediante Maven Surefire. Nel modulo `storm-server`, la selezione comprende le suite manuali, LLM e Randoop, mentre i test EvoSuite non integrabili vengono esclusi tramite il pattern `!*_ESTest`.

La pipeline è stata utilizzata come controllo incrementale durante l'intero sviluppo. Le otto esecuzioni mostrate nella Figura 16 documentano la validazione progressiva delle principali attività: Category Partition, Control-Flow Testing, Mutation Testing, generazione Randoop e LLM e integrazione delle classi rifattorizzate. Le run completate con successo confermano che ogni incremento è stato verificato in un ambiente pulito prima di procedere alla fase successiva.

La CI/CT ha inoltre rilevato un errore di integrazione che non era emerso nelle esecuzioni locali parziali. Dopo il push delle varianti rifattorizzate, alcune classi che avrebbero dovuto essere temporaneamente disabilitate erano rimaste incluse nella selezione Maven, causando il fallimento del workflow. La run negativa ha reso immediatamente visibile il problema; dopo la corretta disabilitazione delle classi e la sistemazione dei test coinvolti, l'esecuzione successiva è terminata con successo. Questo episodio dimostra l'utilità concreta della pipeline: senza GitHub Actions, l'errore avrebbe potuto rimanere nascosto eseguendo localmente soltanto singole classi di test.

5.2 Minaccia alla validità: indisponibilità di EvoSuite

L'unica limitazione strumentale che ha ridotto l'ampiezza del confronto è stata l'impossibilità di integrare correttamente **EvoSuite**. Di conseguenza, l'analisi delle suite automatiche è limitata a LLM e Randoop, mentre i risultati EvoSuite sono indicati come **N/A**.

Il plugin Maven EvoSuite 1.0.6 è legato principalmente all'ecosistema Java 8 e richiede componenti legacy, tra cui `tools.jar`, non compatibili con l'ambiente Java 11/17 utilizzato da Apache Storm 2.7.1. Anche il tentativo in modalità stand-alone su artifact legacy ha prodotto file di test non compilabili o non integrabili nel progetto corrente. Per `RedisKeyValueStateIterator` le difficoltà hanno riguardato anche le dipendenze Redis/Jedis e la visibilità dei metodi; per `StatsUtil`, le strutture Thrift e le mappe annidate hanno reso le suite generate inutilizzabili.

Questa limitazione non modifica i risultati misurati per LLM e Randoop, ma ne circoscrive la validità esterna: non è possibile stabilire come EvoSuite avrebbe performato sullo stesso insieme di classi e varianti. La maggiore adeguatezza osservata dei test LLM deve pertanto essere interpretata esclusivamente rispetto a Randoop e nelle condizioni sperimentali adottate.

6 Conclusioni

La sperimentazione ha mostrato che l'efficacia delle tecniche di testing e refactoring automatico dipende fortemente dalla complessità del SUT. La progettazione incrementale delle suite, basata su Category Partition, Control-Flow e Mutation Testing, ha migliorato progressivamente l'adeguatezza dei test. Per `RedisKeyValueStateIterator` sono stati raggiunti il 100% di Instruction Coverage e un Mutation Score del 100%; per `StatsUtil` i risultati sono rimasti più contenuti a causa delle dimensioni della classe e della complessità delle strutture Thrift e delle mappe annidate.

Nel confronto automatico, i test LLM si sono dimostrati più efficaci di Randoop sulla classe complessa. Sulle varianti di `StatsUtil`, il risultato migliore è stato ottenuto da C_2 , con il 78% di Instruction Coverage, il 64% di Branch Coverage e il 51% di Mutation Score. Randoop ha raggiunto il proprio massimo con C_1 , ottenendo rispettivamente il 17%, l'11% e il 6%. EvoSuite non è stato incluso nel confronto perché le suite generate non sono risultate integrabili nell'ambiente Java 11/17 del progetto.

Il refactoring tramite LLM ha eliminato l'unico smell di `RedisKeyValueStateIterator` e ha ridotto quelli di `StatsUtil` da 265 a un intervallo compreso tra 19 e 45. Tutte le varianti finali compilano e superano la suite standard. Inoltre, il caso di C_2 mostra che la riduzione degli smell non implica automaticamente migliori valori di coverage o Mutation Score.

GitHub Actions ha garantito la validazione continua delle modifiche e ha permesso di individuare errori di integrazione non rilevati dalle esecuzioni locali parziali. Nel complesso, l'IA generativa rappresenta un valido supporto per la generazione dei test e il refactoring, ma deve essere affiancata da compilazione, regression testing, analisi delle metriche e supervisione umana. Il refactoring rimane essenziale per la manutenibilità del software, ma non costituisce da solo una garanzia di correttezza o di eliminazione dei difetti.

A Category Partition: RedisKeyValueStateIterator

Questa appendice descrive le categorie e le partizioni utilizzate per testare l'iteratore `RedisKeyValueStateIterator`, focalizzandosi sul suo ciclo di vita e sull'interazione con lo storage Redis.

Categoria A – Dati restituiti da Redis

- **A1:** zero elementi, corrispondente al caso di storage vuoto.
- **A2:** un elemento, corrispondente al caso minimo non vuoto.
- **A3:** più elementi restituiti da Redis.
- **A4:** lista `null`, interpretata come errore o risposta anomala da Redis.
- **A5:** numero di elementi uguale al `chunkSize`, utilizzato come limite superiore del chunk nella Boundary Value Analysis.

Categoria B – Stato della scansione Redis

- **B1:** scansione conclusa, identificata dal cursore finale pari a 0.
- **B2:** scansione non conclusa, identificata da un cursore diverso da 0.
- **B3:** scansione multi-chunk, terminata dopo più letture successive.
- **B4:** chunk vuoto con cursore non finale.

Categoria C – Stato dell'iteratore

- **C1:** nessun elemento disponibile.
- **C2:** un elemento disponibile.
- **C3:** più elementi disponibili.
- **C4:** elementi caricati ma non ancora consumati.
- **C5:** elementi caricati e successivamente consumati.

Categoria D – Dati pendenti locali

- **D1:** iteratori `pendingPrepare` e `pendingCommit` vuoti.

Categoria E – Esito dell'accesso a Redis

- **E1:** il comando `hscan` termina correttamente.
- **E2:** il comando `hscan` solleva un'eccezione di tipo `RuntimeException`.

Categoria F – Gestione della risorsa Redis

- **F1:** il metodo `getInstance` restituisce un oggetto valido.
- **F2:** il metodo `returnInstance` viene chiamato dopo una lettura corretta.
- **F3:** il metodo `returnInstance` viene chiamato obbligatoriamente dopo un'eccezione, grazie alla presenza del blocco `finally`.

B Category Partition: StatsUtil

La presente appendice riporta le categorie e le partizioni utilizzate per la progettazione dei quattordici casi della suite StatsUtilCategoryPartitionTest.

Categoria A – Forma dell'input

- **A1:** input `null`.
- **A2:** input vuoto.
- **A3:** input composto da una sola finestra temporale.
- **A5:** input con una sola stream.
- **A6:** input con più stream.
- **A7:** stessa chiave presente in più mappe.
- **A8:** chiavi presenti solo in una mappa.

Categoria B – Valori numerici

- **B1:** valori positivi.
- **B2:** valore zero.
- **B3:** valori o metriche mancanti.
- **B5:** valori di tipo `Long`.
- **B6:** valori di tipo `Double`.

Categoria C – Medie pesate

- **C1:** media con conteggio positivo.
- **C2:** media con conteggio zero.
- **C5:** più stream nella stessa finestra.

Categoria D – Statistiche Bolt

- **D1:** metrica `executed` presente.
- **D2:** metrica `execute latency` presente.
- **D3:** metrica `process latency` presente.
- **D4:** stream Bolt presente.

Categoria E – Statistiche Spout

- **E1:** metrica `acked` presente.
- **E2:** metrica `complete latency` presente.
- **E3:** stream Spout presente.

Tabelle di progettazione dei test

Category Partition

Tabella 1: Progettazione del Caso di Test (Category Partition) – RedisKeyValueStateIterator

ID Test	Partizioni di input	Output atteso
TC01	A1, B1, C1, D1, E1, F1, F2	Un iteratore vuoto e il corretto rilascio della risorsa Redis al termine della lettura.
TC02	A2, B1, C2, D1, E1, F1, F2	Un iteratore contenente l'unica coppia chiave-valore restituita da Redis, senza ulteriori elementi disponibili, e il corretto rilascio della risorsa.
TC03	A3, A5, C3, D1, E1, F1, F2	Un iteratore contenente tutti gli elementi del chunk pieno e il corretto rilascio della risorsa Redis.
TC04	A4, B1, D1, E1, F1, F2	Un risultato <code>null</code> , in assenza di un chunk precedente, e il corretto rilascio della risorsa Redis.
TC05	E2, F1, F3	La propagazione dell'eccezione sollevata durante l'accesso a Redis e il rilascio della risorsa anche nel flusso eccezionale.
TC06	A1, B1, C1, E1, F2	Un iteratore vuoto e uno stato finale che segnala la conclusione dei dati nello storage.
TC07	A2, B1, C2, C4, C5, E1, F2	Un iteratore contenente un elemento; la fine dei dati rimane falsa prima del consumo e diventa vera dopo il consumo dell'elemento.
TC08	A3, A5, B1, C3, C4, C5, E1, F2	Un iteratore contenente tutti gli elementi del chunk; la fine dei dati rimane falsa finché sono presenti elementi non consumati e diventa vera dopo il loro completo consumo.
TC09	A1, B2, B4, C1, E1, F2	Un iteratore vuoto e uno stato non finale, poiché il cursore indica che la scansione Redis deve proseguire.
TC10	A1, A2, B1, B2, B3, B4, C1, C2, C4, C5, E1, F2	Dopo il primo chunk, un iteratore vuoto e uno stato non finale; dopo il secondo chunk, un iteratore contenente l'elemento restituito e uno stato finale soltanto dopo il suo consumo.

Tabella 2: Progettazione del Caso di Test (Category Partition) – StatsUtil

ID Test	Partizioni di input	Output atteso
TC01	A3, A5, B1, B5	Una mappa contenente, per la finestra considerata, la somma del contatore associato all'unica stream presente.
TC02	A3, A6, B1, B5	Una mappa in cui i contatori delle diverse stream appartenenti alla stessa finestra sono sommati correttamente.
TC03	A3, A5, B2, B5	Una mappa che mantiene la finestra considerata associandola a un totale pari a zero.
TC04	A7, B1, B5	Una mappa in cui i valori associati alla stessa finestra e alla stessa stream, presenti in più mappe di input, sono sommati.
TC05	A8, A6, B1, B5	Una mappa che mantiene separatamente tutte le stream presenti nelle diverse mappe di input, conservando i rispettivi valori.
TC06	A3, A5, C1, B6	Una mappa contenente, per la finestra considerata, la media pesata calcolata sull'unica stream con conteggio positivo.
TC07	A3, A6, C1, C5, B6	Una mappa contenente la media pesata complessiva delle diverse stream appartenenti alla stessa finestra.
TC08	C2, B2, B6	Una mappa contenente una media pari a zero, senza divisioni per zero o risultati numerici non validi.
TC09	D1, D2, D3, C1, B5, B6	Una mappa contenente il conteggio totale delle esecuzioni e i totali pesati delle latenze di esecuzione e di processo del Bolt.
TC10	A1, B3	Una mappa contenente valori aggregati nulli, rappresentati mediante totali e conteggi pari a zero.
TC11	D1, D2, D3, D4, C1, B5, B6	Una mappa organizzata per stream, contenente per ciascuna stream il conteggio delle esecuzioni e i totali pesati delle latenze del Bolt.
TC12	E1, E2, C1, B5, B6	Una mappa contenente il conteggio complessivo delle tuple confermate e il totale pesato della latenza di completamento dello Spout.
TC13	E1, E2, E3, C1, B5, B6	Una mappa organizzata per stream, contenente per ciascuna stream il conteggio delle tuple confermate e il totale pesato della latenza di completamento dello Spout.
TC14	A2	Una mappa risultato vuota.

Control-Flow Testing

Tabella 3: Test Case Design (Control-Flow) – RedisKeyValueStateIterator

ID Test	Metodo/Ramo coperto	Risultato atteso
TC11	<code>decodeKey(byte[])</code>	La chiave binaria viene deserializzata mediante <code>keySerializer</code> e viene restituita la corrispondente chiave decodificata.
TC12	<code>decodeValue(byte[])</code>	Il valore codificato nel formato previsto dalla classe viene deserializzato mediante <code>valueSerializer</code> e restituito nel tipo atteso.
TC13	<code>isTombstoneValue(byte[])</code>	Un valore ordinario, diverso dal tombstone, non viene riconosciuto come marcatore di cancellazione.

Tabella 4: Test Case Design (Control-Flow) – StatsUtil

ID Test	Metodo/Ramo coperto	Risultato atteso
TC15	<code>floatStr(null)</code>	Viene restituita una rappresentazione testuale equivalente al valore numerico zero.
TC16	<code>floatStr(Double)</code>	Il valore decimale viene restituito come stringa formattata con tre cifre decimali, in modo coerente con il locale impostato.
TC17	<code>errorSubset(String)</code>	La stringa viene troncata alla lunghezza massima prevista dal metodo.
TC18	<code>componentType(..., null)</code>	In assenza dell'identificativo del componente viene restituito un risultato <code>null</code> .
TC19	<code>componentType(..., bolt)</code>	Il componente presente nella mappa dei Bolt viene classificato come Bolt.
TC20	<code>componentType(..., spout)</code>	Il componente non presente tra i Bolt e non appartenente al sistema viene classificato come Spout.
TC21	<code>componentType(..., system)</code>	Il componente di sistema viene classificato come Bolt.
TC22	<code>getFilledStats(...)</code>	La lista risultante contiene soltanto gli <code>ExecutorSummary</code> con statistiche valorizzate.
TC23	<code>convertWorkerBeats(...)</code>	La mappa risultante contiene una voce per ciascun executor e conserva le informazioni temporali dell'heartbeat.
TC24	<code>convertExecutorsStats(...)</code>	Le chiavi di tipo <code>ExecutorInfo</code> vengono convertite nelle corrispondenti coppie contenenti il task iniziale e quello finale.
TC25	<code>convertZkWorkerHb(null)</code>	In presenza di un heartbeat nullo viene restituita una mappa vuota.
TC26	<code>convertZkWorkerHb(...)</code>	La mappa risultante conserva l'identificativo della topologia, l'uptime, il tempo dell'heartbeat e le statistiche degli executor.
TC27	<code>thriftifyRpcWorkerHb(...)</code>	Viene creato un heartbeat RPC contenente l'identificativo della topologia, l'executor richiesto e il tempo di creazione.
TC28	<code>aggregateSpoutStreams(...)</code>	Le statistiche Spout vengono aggregate per finestra, producendo i contatori complessivi e la latenza media pesata.
TC29	<code>aggregateBoltStreams(...)</code>	Le statistiche Bolt vengono aggregate per finestra, producendo i contatori complessivi e le latenze medie pesate.
TC30	<code>aggregateAverages(...)</code>	Viene restituita una mappa contenente la media pesata ottenuta combinando più sequenze di medie e conteggi.
TC31	<code>preProcessStreamSummary(..., false)</code>	Gli stream di sistema vengono esclusi dalla struttura risultante, mentre gli stream ordinari vengono mantenuti.
TC32	<code>preProcessStreamSummary(..., true)</code>	La struttura risultante conserva sia gli stream ordinari sia gli stream di sistema.
TC33	<code>extractNodeInfosFromHbForComp(.La)</code>	La lista risultante contiene esclusivamente le informazioni relative a host e porta del componente richiesto.

Mutation Testing

Tabella 5: Test Case Design (Mutation Testing) – RedisKeyValueStateIterator

ID Test	Mutazione target	Risultato atteso
MT01	isTombstoneValue: valore di ritorno sostituito con <i>false</i>	Un valore uguale al tombstone viene riconosciuto come marcatore di cancellazione; il test fallisce se il metodo restituisce sempre <i>false</i> .

Tabella 6: Test Case Design (Mutation Testing) – StatsUtil

ID	Metodo target	Mutazione target	Risultato atteso
MT01	aggBoltStreamsLatAndCount	Sostituzione della mappa vuota modificabile restituita con una struttura vuota non modificabile.	La mappa restituita con input nulli è vuota ma consente l'inserimento di nuovi elementi.
MT02	aggSpoutStreamsLatAndCount	Sostituzione della mappa vuota modificabile con una struttura equivalente ma non modificabile.	La mappa restituita con input nulli è vuota ma consente l'inserimento di nuove statistiche.
MT03	spoutStreamsStats	Sostituzione del risultato vuoto con una mappa non modificabile quando la lista di statistiche è nulla.	Il metodo restituisce una mappa vuota nella quale è possibile inserire nuove categorie di statistiche.
MT04	boltStreamsStats	Sostituzione del risultato vuoto con una mappa non modificabile quando la lista di statistiche è nulla.	Il metodo restituisce una mappa vuota nella quale è possibile inserire nuove categorie di statistiche.
MT05	convertZkExecutorHb	Sostituzione della mappa vuota modificabile restituita per un heartbeat nullo.	Con un heartbeat nullo viene restituita una mappa vuota e modificabile.
MT06	convertZkWorkerHb	Sostituzione della mappa vuota modificabile restituita per un heartbeat nullo.	Con un heartbeat nullo viene restituita una mappa vuota e modificabile.
MT07	extractDataFromHb	Sostituzione della lista vuota modificabile con una lista vuota non modificabile.	Con input nullo viene restituita una lista vuota nella quale è possibile aggiungere nuovi elementi.
MT08	aggSpoutLatAndCount	Alterazione della somma dei conteggi acknowledged o del calcolo del totale pesato della complete latency.	Il risultato contiene la somma dei conteggi e il totale delle latenze pesato mediante i rispettivi valori di acknowledged.
MT09	aggBoltLatAndCount	Alterazione della somma di executed o dei totali pesati delle latenze di esecuzione e di processo.	Il risultato contiene il totale delle esecuzioni e i totali delle latenze calcolati mediante i rispettivi conteggi.
MT10	aggBoltStreamsLatAndCount	Alterazione dell'aggregazione per stream dei conteggi e delle latenze totali del Bolt.	La stream viene mantenuta e contiene il conteggio delle esecuzioni e i totali pesati delle due latenze.
MT11	aggBoltStreamsLatAndCount	Inserimento improprio della processLatencyTotal quando la mappa delle latenze di processo è nulla.	Il totale della latenza di esecuzione viene calcolato, mentre la metrica della latenza di processo non viene inserita.
MT12	aggSpoutStreamsLatAndCount	Alterazione del conteggio acknowledged o del totale pesato della latenza associata alla singola stream.	La stream viene mantenuta con il conteggio delle tuple confermate e il totale pesato della latenza di completamento.
MT13	aggregateCountStreams	Alterazione della somma dei contatori appartenenti a stream della stessa finestra.	I contatori delle stream vengono sommati e associati alla relativa finestra temporale.
MT14	aggregateCounts	Alterazione della somma quando la stessa finestra e la stessa stream sono presenti in più mappe.	I valori relativi alla stessa chiave vengono sommati, mentre le stream presenti in una sola mappa vengono mantenute.
MT15	aggregateAvgStreams	Alterazione del calcolo della media pesata tra più stream della stessa finestra.	La media risultante viene calcolata pesando ciascun valore con il corrispondente conteggio.
MT16	aggregateAvgStreams	Gestione errata di una stream dotata di conteggio ma priva della corrispondente media.	Alla media mancante viene associato il valore zero e il risultato complessivo viene calcolato usando tutti i conteggi.

Continua nella pagina successiva

ID	Metodo target	Mutazione target	Risultato atteso
MT17	aggregateAverages	Alterazione della combinazione pesata di più sequenze di medie e conteggi.	Le medie delle diverse sequenze vengono combinate usando i rispettivi conteggi come pesi.
MT18	aggregateAverages	Sostituzione della mappa vuota modificabile restituita per sequenze vuote.	Con sequenze vuote viene restituita una mappa vuota nella quale è possibile inserire nuovi elementi.
MT19	aggregateSpoutStreams	Alterazione dell'aggregazione complessiva dei contatori Spout o della media pesata della latenza di completamento.	Il risultato contiene i totali di <code>acked</code> , <code>failed</code> , <code>emitted</code> e <code>transferred</code> , oltre alla latenza media pesata.
MT20	aggregateBoltStreams	Alterazione dell'aggregazione complessiva dei contatori Bolt o delle medie pesate delle latenze.	Il risultato contiene tutti i contatori aggregati e le latenze di processo ed esecuzione pesate con i rispettivi conteggi.
MT21	preProcessStreamSummary	Mancata rimozione degli stream di sistema quando <code>includeSys</code> è impostato a <code>false</code> .	Gli stream ordinari vengono mantenuti, mentre quelli di sistema vengono rimossi dalle statistiche emesse e trasferite.
MT22	mergeAggCompStatsTopology	Alterazione delle somme dei campi Bolt o della selezione della capacità massima.	I campi numerici vengono sommati, il numero degli <code>executor</code> viene incrementato e viene mantenuta la capacità massima.
MT23	mergeAggCompStatsTopology	Alterazione della somma dei conteggi e dei totali di latenza delle statistiche Spout.	I campi numerici delle due statistiche vengono sommati e il numero degli <code>executor</code> viene incrementato.
MT24	mergeAggCompStatsTopology	Gestione alterata di valori numerici non validi, come <code>NaN</code> , nei campi aggregati.	I valori numerici non validi vengono normalizzati al valore zero secondo il comportamento previsto dal metodo.
MT25	mergeAggCompStatsCompStats	Alterazione del calcolo delle latenze medie dell' <code>executor</code> Bolt a partire dai totali e dai conteggi.	Le statistiche dell' <code>executor</code> contengono il conteggio delle esecuzioni e le latenze medie correttamente calcolate.
MT26	mergeAggCompStatsCompStats	Alterazione del calcolo della latenza media di completamento dell' <code>executor</code> Spout.	Le statistiche dell' <code>executor</code> contengono il conteggio <code>acked</code> e la relativa latenza media di completamento.
MT27	extractNodeInfosFromHb	Mancato filtraggio delle informazioni host e porta in base al componente richiesto.	La lista contiene soltanto host e porta associati al componente selezionato.
MT28	componentType	Alterazione dei valori restituiti nei rami relativi a componente nullo, Bolt e Spout.	Un identificativo nullo produce <code>null</code> ; i componenti Bolt e Spout vengono classificati con il rispettivo tipo.
MT29	aggWorkerStats	Alterazione del conteggio degli <code>executor</code> o dei task associati ai componenti del worker.	La statistica risultante contiene il numero corretto di <code>executor</code> , i task raggruppati per componente e il proprietario della topologia.
MT30	convertWorkerBeats	Alterazione della conversione degli <code>executor</code> presenti nell' <code>heartbeat</code> o perdita dell'informazione temporale.	Viene prodotta una voce per ciascun <code>executor</code> e viene mantenuto il tempo dell' <code>heartbeat</code> .
MT31	convertExecutorsStats	Alterazione della conversione della chiave <code>ExecutorInfo</code> nella coppia di task.	La mappa risultante usa come chiave la coppia composta dal task iniziale e da quello finale e mantiene le statistiche associate.
MT32	thriftifyRpcWorkerHb	Alterazione dell'identificativo della topologia o dei limiti dell' <code>executor</code> nell' <code>heartbeat</code> RPC.	L' <code>heartbeat</code> contiene l'identificativo della topologia e un <code>executor</code> con i limiti iniziale e finale richiesti.

Materiale visivo di supporto

Evidenze Javadoc

```
org.apache.storm.redis.state.RedisKeyValueStateIterator
protected Iterator<Entry<byte[], byte[]>> loadChunkFromStateStorage()
Load some part of state KV's from storage and returns iterator of cached data from storage.
Returns:
    Iterator of loaded state KV's
Iterator<Entry<byte[], byte[]>> org.apache.storm.redis.state.RedisKeyValueStateIterator.loadChunkFromStateStorage()
Source: storm-redis
loadChunkFromStateStorage() {
```

Figura 1: Javadoc di `loadChunkFromStateStorage`.

```
org.apache.storm.redis.state.RedisKeyValueStateIterator
protected boolean isEndOfDataFromStorage()
Check whether end of data is reached from storage state KVs.
Returns:
    whether end of data is reached from storage state KVs

boolean org.apache.storm.redis.state.RedisKeyValueStateIterator.isEndOfDataFromStorage()
Source: storm-redis
```

Figura 2: Javadoc di `isEndOfDataFromStorage`.

```
org.apache.storm.stats.StatsUtil

public static <K> Map<String, Double> aggregateAvgStreams(Map<String, Map<K, Double>> avg,
                                                         Map<String, Map<K, Long>> counts)

aggregate weighted average of all streams.

Parameters:
    avg - a Map of {key -> stream -> average value}
    counts - a Map of {key -> stream -> count value}
Returns: Map of {key -> aggregated value}
```

Figura 3: Javadoc sulle medie pesate.

```
org.apache.storm.state:storm-state
```

```
public static Map<String, Number> aggBoltListAndCount(Map<List<String>, Double> id2ExecAvg,
Map<List<String>, Double> id2ProcAvg,
Map<List<String>, Long> id2NumExec)
```

Aggregates number executed, process latency, and execute latency across all streams.

Parameters:

- id2ExecAvg - [global stream id -> exec avg value] e.g., ["split"."default", 0.44313]
- id2ProcAvg - [global stream id -> proc avg value]
- id2NumExec - [global stream id -> executed]

Figura 4: Javadoc di `aggBoltLatAndCount`.

```
org.apache.storm.stats.StatsUtil  
public static Map<String, Number> aggSpoutLatAndCount(Map<String, Double> id2compAvg,  
                                                       Map<String, Long> id2numAcked)  
aggregate number acked and complete latencies across all streams.
```

Figura 5: Javadoc di `aggSpoutLatAndCount`.

Report JaCoCo

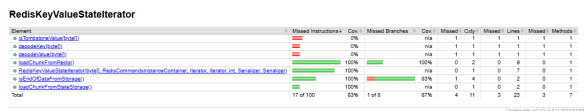


Figura 6: JaCoCo dopo Category Partition per RedisKeyValueStateIterator.

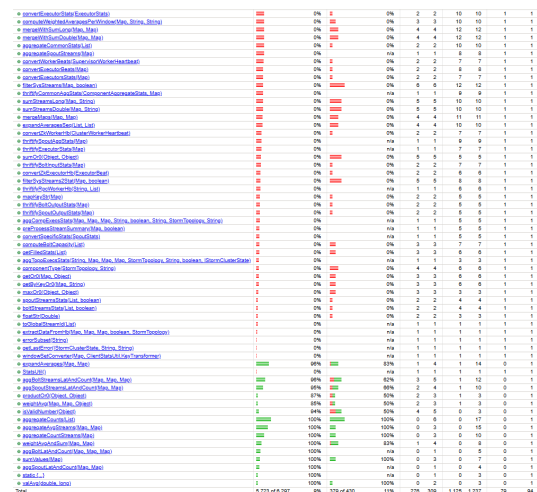


Figura 7: JaCoCo dopo Category Partition per StatsUtil.

Report PIT

Rapporto sulla copertura dei test nei box

Riassunto del progetto

Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
1	88% 21/24	67% 8/12	100% 8/8

Suddivisione per pacchetto

Nome	Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
org.apache.storm.redis.state 1		88% 21/24	67% 8/12	100% 8/8

- JUnit 5 è sul percorso di classe ma il plugin pitest:junit5 non è installato.

Supporto del report generato da [PIT](#) 1.25.7

Funzionalità migliorate disponibili su arcmutate.com

Figura 8: PIT Category Partition per RedisKeyValueStateIterator.

Rapporto sulla copertura dei test nei box

Riassunto del progetto

Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
1	100% 24/24	92% 11/12	92% 11/12

Suddivisione per pacchetto

Nome	Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
org.apache.storm.redis.state.1	100%	24/24	92%	11/12

- JUnit 5 è sul percorso di classe ma il plugin pitest junit 5 non è installato

Supporto del report generato da [PIT 1.25.7](#)

Funzionalità migliorate disponibili su arcmutate.com

Figura 9: PIT Category Partition e Control-Flow per RedisKeyValueStateIterator.

Rapporto sulla copertura dei test nei box

Riepilogo del pacchetto

org.apache.storm.stats

Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
1	9%112/1237	10%42/413	93%42/45

Suddivisione per classe

Nome	Copertura della linea	Copertura delle mutazioni	Forza del test
StatsUtil.java	9%112/1237	10%42/413	93%42/45

Report generato dal supporto PIT 1.25.7

Figura 10: PIT Category Partition per StatsUtil.

Mutazioni	
12	1. chiamata rimossa a org.apache.storm.redis.state.RedisKeyValueStateIterator::isNotEmptyFromRedis = KILLED
13	1. sostituito il valore di ritorno con null per org.apache.storm.redis.state.RedisKeyValueStateIterator::isNotEmptyFromRedis = KILLED
14	1. Negato condizionale = UC350
15	1. sostituito il ritorno booleano con true per org.apache.storm.redis.state.RedisKeyValueStateIterator::isNotEmptyFromRedis = KILLED
16	1. Negato condizionale = UC350
17	1. sostituito il valore di ritorno con null per org.apache.storm.redis.state.RedisKeyValueStateIterator::decodeKey = KILLED
18	1. sostituito return value con null per org.apache.storm.redis.state.RedisKeyValueStateIterator::decodeValue = KILLED
19	1. sostituito il ritorno booleano con true per org.apache.storm.redis.state.RedisKeyValueStateIterator::isTombstoneValue = KILLED
20	1. sostituito il ritorno booleano con false per org.apache.storm.redis.state.RedisKeyValueStateIterator::isTombstoneValue = SOPRAVVISUTO Copertura del test
21	1. Negato condizionale = UC350
22	1. chiamata rimossa a org.apache.storm.redis.common.container.RedisCommandsInstanceContainer::returnInstance = UC350

Figura 12: Mutante sopravvissuto su isTombstoneValue.

Rapporto sulla copertura dei test nei box

Riassunto del progetto

Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
1	21%262/1237	23%94/413	91%94/103

Suddivisione per pacchetto

Nome	Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
org.apache.storm.stats	1	21%262/1237	23%94/413	91%94/103

Supporto del report generato da PIT 1.25.7

Funzionalità migliorate disponibili su arcmutate.com

Figura 11: PIT Category Partition e Control-Flow per StatsUtil.

Rapporto sulla copertura dei test nei box

Riassunto del progetto

Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
1	100%24/24	100%12/12	100%12/12

Suddivisione per pacchetto

Nome	Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
org.apache.storm.redis.state	1	100%24/24	100%12/12	100%12/12

- Il Unit 5 è sul percorso di classe ma il plugin pitest junit 3 non è installato.

Supporto del report generato da PIT 1.25.7

Funzionalità migliorate disponibili su arcmutate.com

Figura 13: PIT finale per RedisKeyValueStateIterator.

Rapporto sulla copertura dei test nei box

Riassunto del progetto

Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
1	37%463/1237	38%156/413	81%156/193

Suddivisione per pacchetto

Nome	Numero di classi	Copertura della linea	Copertura delle mutazioni	Forza del test
org.apache.storm.stats	1	37%463/1237	38%156/413	81%156/193

Supporto del report generato da PIT 1.25.7

Funzionalità migliorate disponibili su arcmutate.com

Figura 14: PIT finale per StatsUtil.

Evidenze EvoSuite

```
1 package org.apache.storm.redis.state;
2
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5 import static org.evosuite.shaded.org.mockito.Mockito.*;
6 import static org.evosuite.runtime.EvoAssertions.*;
7
8 import java.util.Iterator;
9 import java.util.Map;
10 import org.apache.storm.redis.common.container.RedisCommandsInstanceContainer;
11 import org.apache.storm.redis.state.RedisKeyValueStateIterator;
12 import org.apache.storm.state.Serializer;
13 import org.evosuite.runtime.EvoRunner;
14 import org.evosuite.runtime.EvoRunnerParameters;
15 import org.evosuite.runtime.ViolatedAssumptionAnswer;
16 import org.junit.runner.RunWith;
17
18 @RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS = true, useNET = true, resetStaticState = true, separateClassLoader = true)
19 public class RedisKeyValueStateIterator_ESTest extends RedisKeyValueStateIterator_ESTest_scaffolding {
20
21     @Test(timeout = 4000)
22     public void test0() throws Throwable {
23         Iterator<Map.Entry<byte[], byte[]>> iterator0 = (Iterator<Map.Entry<byte[], byte[]>>) mock(Iterator.class, new ViolatedAssumptionAnswer());
24         byte[] byteArray0 = new byte[1];
25         RedisKeyValueStateIterator<Integer, Integer> redisKeyValueStateIterator0 = null;
26         try {
27             redisKeyValueStateIterator0 = new RedisKeyValueStateIterator<Integer, Integer>(byteArray0, (RedisCommandsInstanceContainer) null, iterator0, iterator0,
28                 fail("Expecting exception: NoClassDefFoundError"));
29         } catch (NoClassDefFoundError e) {
30             //
31             // Could not initialize class org.apache.storm.state.DefaultStateEncoder
32             //
33             verifyException("org.apache.storm.redis.state.RedisKeyValueStateIterator", e);
34         }
35     }
36 }
```

Figura 15: Test generato da EvoSuite per RedisKeyValueStateIterator. Il file è stato prodotto, ma non è risultato compilabile nell'ambiente Apache Storm 2.7.1.

C Prompt utilizzati per la generazione tramite LLM

Questa appendice riporta gli schemi di prompt utilizzati per generare automaticamente test tramite Microsoft Copilot. Sono state adottate tre strategie: *zero-shot*, *few-shot* e *guided Tree-of-Thought*. Per ogni strategia il prompt è stato adattato alle due classi oggetto di studio: `RedisKeyValueStateIterator` e `StatsUtil`.

Zero-shot Prompting

Con lo zero-shot prompting non sono stati forniti esempi al modello. Il prompt contiene solo il ruolo assegnato al modello, il nome della classe, il nome atteso del file di test e il codice sorgente da analizzare.

RedisKeyValueStateIterator

```
As a professional software tester who writes Java test methods, generate a complete JUnit 4 test file named RedisKeyValueStateIteratorZeroShotTest.java for the Java class RedisKeyValueStateIterator. The test file must belong to the package org.apache.storm.redis.state. Generate meaningful unit tests for the observable behavior of the class, including normal cases, boundary cases, and error scenarios. Here is the source code of RedisKeyValueStateIterator: {source_code}.
```

StatsUtil

```
As a professional software tester who writes Java test methods, generate a complete JUnit 4 test file named StatsUtilZeroShotTest.java for the Java class StatsUtil. The test file must belong to the package org.apache.storm.stats.llm and must test the public static methods of org.apache.storm.stats.StatsUtil. Generate meaningful unit tests for normal cases, boundary cases, and error scenarios. Here is the source code of StatsUtil: {source_code}.
```

Few-shot Prompting

Con il few-shot prompting sono stati forniti al modello alcuni esempi di test manuali, in modo da guidare lo stile degli assert, la struttura della classe di test e la scelta dei casi limite.

RedisKeyValueStateIterator

```
As a professional software tester who writes Java test methods, consider the following examples of manually written tests for RedisKeyValueStateIterator: {fewshot_example}. Now generate a complete JUnit 4 test file named RedisKeyValueStateIteratorFewShotTest.java for the class RedisKeyValueStateIterator. The test file must belong to the package org.apache.storm.redis. It should comprehensively test the behavior of the iterator, including Redis scan results, cursor handling, resource release, key/value decoding, and tombstone handling. Your output file must start with ###Test START## and finish with ###Test END##. Here is the source code of RedisKeyValueStateIterator: {source_code}.
```

StatsUtil

```
As a professional software tester who writes Java test methods, consider the following examples of manually written tests for StatsUtil: {fewshot_example}. Now generate a complete JUnit 4 test file named StatsUtilFewShotTest.java for the class StatsUtil. The test file must belong to the package org.apache.storm.stats.llm and must test the public static methods of org.apache.storm.stats.StatsUtil. Focus on counter aggregation, weighted averages, Bolt/Spout statistics, stream filtering, component classification, and conversion utilities. Your
```

output file must start with `###Test START##` and finish with `###Test END##`. Here is the source code of StatsUtil: {source_code}.

Guided Tree-of-Thought Prompting

Con il guided Tree-of-Thought il modello è stato guidato in modo più strutturato. Il prompt richiede di simulare più esperti di testing, identificare i metodi, proporre casi base, individuare edge case e infine produrre una suite completa.

RedisKeyValueStateIterator

Imagine three different experts in software testing who are tasked with developing comprehensive JUnit 4 test cases for the Java class RedisKeyValueStateIterator. The final output must be a complete test file named RedisKeyValueStateIteratorCoTTest.java in the package org.apache.storm.redis.

1. Extract and list the relevant methods of RedisKeyValueStateIterator, including protected methods that can be tested from the same package.
2. For each method, generate a basic JUnit 4 test case that checks the expected functionality.
3. Identify edge cases related to empty Redis results, one element, full chunks, non-final cursors, null scan results, exceptions, key/value decoding, and tombstone values.
4. Generate JUnit 4 test cases for the identified edge cases and exception scenarios, using Mockito where external Redis dependencies are required.
5. Merge all test cases into one complete JUnit 4 test file named RedisKeyValueStateIteratorCoTTest.java.

The complete JUnit test file must start with `###Test START##` and finish with `###Test END##`. Here is the source code of RedisKeyValueStateIterator: {source_code}.

StatsUtil

Imagine three different experts in software testing who are tasked with developing comprehensive JUnit 4 test cases for the Java class StatsUtil. The final output must be a complete test file named StatsUtilCoTTest.java in the package org.apache.storm.stats.llm. The tests must target org.apache.storm.stats.StatsUtil.

1. Extract and list the main public static methods of StatsUtil, including methods for aggregation, weighted averages, Bolt/Spout statistics, stream filtering, component classification, heartbeat conversion, and formatting utilities.
2. For each selected method, generate a basic JUnit 4 test case that checks the normal behavior.
3. Given the source code, identify edge cases involving null inputs, empty maps, nested maps, zero counts, missing values, system streams, Thrift objects, and invalid numeric values.
4. Generate JUnit 4 test cases that specifically test the identified edge cases and exception scenarios.
5. Merge all individual test cases into one complete JUnit 4 test file named StatsUtilCoTTest.java.

The complete JUnit test file must start with `###Test START##` and finish with `###Test END##`. Here is the source code of StatsUtil: {source_code}.

Tabella 7: Risultati LLM – RedisKeyValueStateIterator

Prompt	Instruction Coverage	Branch Coverage	Mutation Coverage	Test Strength
Zero-shot	100%	100%	92% 11/12	92%
Few-shot	100%	100%	92% 11/12	92%
Guided Tree-of-Thought	100%	100%	92% 11/12	92%

Tabella 8: Risultati LLM – StatsUtil

Prompt	Instruction Coverage	Branch Coverage	Mutation Coverage	Test Strength
Zero-shot	23%	30%	27% 113/413	83%
Few-shot	23%	31%	28% 116/413	86%
Guided Tree-of-Thought	23%	31%	28% 116/413	86%

Tabelle del refactoring automatico

Tabella 9: Compilabilità e superamento della suite standard

Classe	Variante	Compila	Suite standard
RedisKeyValueStatellocator	C_1	Sì	Pass
RedisKeyValueStatellocator	C_2	Sì	Pass
RedisKeyValueStatellocator	C_3	Sì	Pass
RedisKeyValueStatellocator	C_4	Sì	Pass
StatsUtil	C_1	Sì	Pass
StatsUtil	C_2	Sì	Pass
StatsUtil	C_3	Sì	Pass
StatsUtil	C_4	Sì	Pass

Tabella 10: Numero di code smell prima e dopo il refactoring

Classe	C_0	C_1	C_2	C_3	C_4
RedisKeyValueStatellocator	1	0	0	0	0
StatsUtil	265	19	45	19	19

Tabella 11: Confronto dei delta rispetto a C_0 per RedisKeyValueStatellocator

Confronto		C_1			C_2			C_3			C_4		
Baseline	Suite su C_0	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES
C_0	LLM CoT	$\Delta I = 0,00$ p.p.			$\Delta I = 0,00$ p.p.			$\Delta I = 0,00$ p.p.			$\Delta I = 0,00$ p.p.		
		$\Delta B = -12,50$ p.p.			$\Delta B = -12,50$ p.p.			$\Delta B = -12,50$ p.p.			$\Delta B = -12,50$ p.p.		
		$\Delta M = +8,33$ p.p.	-	-	$\Delta M = +8,33$ p.p.	-	-	$\Delta M = +8,33$ p.p.	-	-	$\Delta M = +8,33$ p.p.	-	-
	Randoop	$\Delta I = 0,00$ p.p.			$\Delta I = 0,00$ p.p.			$\Delta I = 0,00$ p.p.			$\Delta I = 0,00$ p.p.		
		$\Delta B = 0,00$ p.p.			$\Delta B = 0,00$ p.p.			$\Delta B = 0,00$ p.p.			$\Delta B = 0,00$ p.p.		
	EvoSuite	-	-	N/A	-	-	N/A	-	-	N/A	-	-	N/A

ΔI : Instruction Coverage; ΔB : Branch Coverage; ΔM : Mutation Score; RND: Randoop; ES: EvoSuite.

Tabella 12: Confronto dei delta rispetto a C_0 per StatsUtil

Confronto		C_1			C_2			C_3			C_4		
Baseline	Suite su C_0	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES	Test LLM	Test RND	Test ES
C_0	LLM CoT	$\Delta I = +32$ p.p.			$\Delta I = +55$ p.p.			$\Delta I = +14$ p.p.			$\Delta I = +2$ p.p.		
		$\Delta B = +23$ p.p.			$\Delta B = +33$ p.p.			$\Delta B = +11$ p.p.			$\Delta B = +1$ p.p.		
		$\Delta M = +22$ p.p.	-	-	$\Delta M = +23$ p.p.	-	-	$\Delta M = +8$ p.p.	-	-	$\Delta M = 0$ p.p.	-	-
	Randoop		$\Delta I = +10$ p.p.			$\Delta I = +1$ p.p.			$\Delta I = 0$ p.p.			$\Delta I = +1$ p.p.	
			$\Delta B = +7$ p.p.			$\Delta B = +1$ p.p.			$\Delta B = +1$ p.p.			$\Delta B = +1$ p.p.	
			$\Delta M = +6$ p.p.	-	-	$\Delta M = +3$ p.p.	-	-	$\Delta M = +3$ p.p.	-	-	$\Delta M = +3$ p.p.	-
	EvoSuite	-	-	N/A	-	-	N/A	-	-	N/A	-	-	N/A

ΔI : Instruction Coverage; ΔB : Branch Coverage; ΔM : Mutation Score; RND: Randoop; ES: EvoSuite.

D Prompt utilizzati per il refactoring tramite LLM

Variante C_1 : analisi statica

You are an expert Java developer. I want to improve the maintainability of the attached C0 class. Create C1 without changing C0 functionality and by removing the following smells reported by SonarCloud: [elenco smell]. Do not include changes different from what I asked. C1 must be a drop-in replacement for C0, with the same package and public method signatures. Here is the source code: {source_code}.

Variante C_2 : vincoli funzionali Black-Box

You are an expert Java developer. Create C2 by removing the following smells: [elenco smell]. Make sure C2 passes the following functional tests designed with Category Partition: [test Black-Box]. Use these tests as specification oracles. Do not change public APIs or signatures. C2 must be a drop-in replacement for C0. Here is the source code: {source_code}.

Variante C_3 : vincoli Control-Flow

You are an expert Java developer. Create C3 by removing the smells: [elenco smell]. In addition to passing the previous tests, consider the following JaCoCo coverage information: [Instruction e Branch Coverage]. Do not alter complex branches or edge cases identified by the report. The code must remain fully compatible with the system. Here is the source code: {source_code}.

Variante C_4 : vincoli di Mutation Testing

You are an expert Java developer. Create C4 by removing the smells: [elenco smell]. The code must pass all previous tests and satisfy the following PIT Mutation Testing constraints: [report PIT]. Ensure that the mutation oracles remain capable of detecting behavioral changes. C4 must be a perfect drop-in replacement for C0, without signature changes. Here is the source code: {source_code}.

8 workflow runs		Event ▾	Status ▾	Branch ▾	Actor ▾
✓	risoluzione fallimento action rendendo disable classi refactored Milestone 4 - Testing & Refactoring CI #8: Commit 2bd59db pushed by enricobarbatano	milestone4-refactoring	6 minutes ago 5m 42s	...	
✗	risoluzione problematiche test randoop rilevate tramite github acti... Milestone 4 - Testing & Refactoring CI #7: Commit f7fe0d5 pushed by enricobarbatano	milestone4-refactoring	11 minutes ago 1m 46s	...	
✗	definizione classi refattorizzate e nuovi test llm e randoop Milestone 4 - Testing & Refactoring CI #6: Commit 2a9f24c pushed by enricobarbatano	milestone4-refactoring	Today at 20:14 5m 57s	...	
✓	definizione test randoop, evosuite e LLM tramite vari prompt Milestone 4 - Testing & Refactoring CI #5: Commit 0e7d651 pushed by enricobarbatano	milestone4-refactoring	Jul 10, 23:32 GMT+2 6m 22s	...	
✓	definizione test randoop Milestone 4 - Testing & Refactoring CI #4: Commit 2dc25ec pushed by enricobarbatano	milestone4-refactoring	Jul 10, 18:52 GMT+2 4m 46s	...	
✓	progettazione ed implementazione dei mutation step con l'obittiv... Milestone 4 - Testing & Refactoring CI #3: Commit b514fa4 pushed by enricobarbatano	milestone4-refactoring	Jul 10, 18:14 GMT+2 5m 40s	...	
✓	implementazione test control flow allo scopo di aumentare l'adequ... Milestone 4 - Testing & Refactoring CI #2: Commit 3c3cf1e pushed by enricobarbatano	milestone4-refactoring	Jul 10, 17:22 GMT+2 5m 2s	...	
✓	progettazione ed implementazione test category partition Milestone 4 - Testing & Refactoring CI #1: Commit dab7b75 pushed by enricobarbatano	milestone4-refactoring	Jul 10, 16:27 GMT+2 5m 20s	...	

Figura 16: Esecuzioni del workflow GitHub Actions associate alle principali fasi di progettazione, implementazione e integrazione. Le run fallite hanno consentito di individuare classi rifattorizzate rimaste abilitate e problemi nei test; le esecuzioni successive documentano la correzione e la validazione della configurazione finale.